Министерство науки и высшего образования Российской Федерации

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

РАЗРАБОТКА АЛГОРИТМОВ АВТОМАТИЧЕСКОГО ПОДБОРА ПАРАМЕТРОВ SAT И SMT РЕШАТЕЛЕЙ В ПРИМЕНЕНИИ К ЗАДАЧАМ СИМВОЛЬНОГО ИСПОЛНЕНИЯ ПРОГРАММ

Автор: Косогоров Евгений Михайлович _____

Направление подготовки: 01.03.02 Прикладная математика и информатика

Квалификация: Бакалавр

Руководитель ВКР: Семенов А.А., кандидат технических наук _____

Санкт-Петербург, 2024 г.

Обучающийся Косогоров Евгений Михайлович
Группа М34391 Факультет ИТиП

Направленность (профиль), специализация
Информатика и программирование

Консультанты:

а) Чухарев К.И., аспирант, ФИТиП _____

ВКР принята «____» _____ 20___ г.

Оригинальность ВКР _____%

ВКР выполнена с оценкой _____

Дата защиты «____» _____ 20___ г.

Секретарь ГЭК Штумпф С.А. _____

Листов хранения _____

Демонстрационных материалов/Чертежей хранения _____

Министерство науки и высшего образования Российской Федерации

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

УТВЕРЖДАЮ

Руководитель ОП

проф., д.т.н. Парфенов В.Г. _____

«____» _____ 20___ г.

ЗАДАНИЕ
НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ

Обучающийся Косогоров Евгений Михайлович

Группа M34391 **Факультет** ИТиП

Квалификация: Бакалавр

Направление подготовки: 01.03.02 Прикладная математика и информатика

Направленность (профиль) образовательной программы: Информатика и программирование

Тема ВКР: Разработка алгоритмов автоматического подбора параметров SAT и SMT решателей в применении к задачам символьного исполнения программ

Руководитель Семенов А.А., кандидат технических наук, Университет ИТМО, факультет информационных технологий и программирования, старший научный сотрудник

2 Срок сдачи студентом законченной работы до: «31» мая 2024 г.

3 Техническое задание и исходные данные к работе

4 Содержание выпускной квалификационной работы (перечень подлежащих разработке вопросов)

5 Перечень графического материала (с указанием обязательного материала)

Графические материалы и чертежи работой не предусмотрены

6 Исходные материалы и пособия

7 Дата выдачи задания «09» октября 2023 г.

Руководитель ВКР _____

Задание принял к исполнению _____ «09» октября 2023 г.

Министерство науки и высшего образования Российской Федерации

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

АННОТАЦИЯ
ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЫ

Обучающийся: Косогоров Евгений Михайлович

Наименование темы ВКР: Разработка алгоритмов автоматического подбора параметров SAT и SMT решателей в применении к задачам символьного исполнения программ

Наименование организации, в которой выполнена ВКР: Университет ИТМО

ХАРАКТЕРИСТИКА ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЫ

1 Цель исследования: Разработать алгоритмы параметризации SAT и SMT решателей, ускорить решения SMT запросов, связанных с символьным исполнение программ, за счёт применения разработанных алгоритмов

2 Задачи, решаемые в ВКР:

а) Исследование применения алгоритмов параметризации SAT и SMT решателей к задачам символьного исполнения программ
б) Разработка методов построения функций оптимизации, учитывающих особенности подклассов решаемых SMT зарпосов
в) Ускорение работы решателей на задачах символьного исполнения программ
г) Программная реализация алгоритмов
д) Проведение вычислительных экспериментов и анализ результатов

3 Число источников, использованных при составлении обзора: 0

4 Полное число источников, использованных в работе: 17

5 В том числе источников по годам:

Отечественных			**Иностранных**		
Последние 5 лет	От 5 до 10 лет	Более 10 лет	Последние 5 лет	От 5 до 10 лет	Более 10 лет
0	0	17	0	0	0

6 Использование информационных ресурсов Internet: да, число ресурсов: 11

7 Использование современных пакетов компьютерных программ и технологий: нет

8 Краткая характеристика полученных результатов

Использование конфигурация параметров, подобранная разработанными алгоритмами параметризации, уменьшило время работы решателей на задачах символьного исполнения программ

9 Гранты, полученные при выполнении работы

10 Наличие публикаций и выступлений на конференциях по теме выпускной работы

Обучающийся Косогоров Е.М. _____

Руководитель ВКР Семенов А.А. _____

«____» _____ 20___ г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 5
1. Символьное исполнение программ и задача подбора параметров 7
1.1. Задача булевой выполнимости 7
1.2. SAT решатели 8
1.3. Задача выполнимости в теориях 10
1.4. SMT решатели 11
1.5. Символьное исполнение 14
1.6. Задача подбора параметров 17
1.7. Цель и задачи работы 19
Выводы по главе 1 20
2. Разработка методов подбора параметров и построение целевых функций 21
2.1. Метод Монте-Карло для вычисления целевой функции 21
2.2. Сведение к задаче псевдобулевой оптимизации 24
2.3. Описание эволюционных стратегий 25
2.4. (1 + 1)-ЭА 27
2.5. Гибридный алгоритм 29
2.6. Байесовская оптимизация 30
2.7. Hill Climbing и MVP 31
Выводы по главе 2 33
3. Вычислительные эксперименты 34
3.1. SMT решатель 34
3.2. Тестовые классы 34
3.3. Определение целевой функции 35
3.4. Конфигурация вычислительной платформы 36
3.5. Постановка экспериментов 36
Выводы по главе 3 39
ЗАКЛЮЧЕНИЕ 40
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 41

ВВЕДЕНИЕ

Символьное исполнение программ (symbolic execution) – это метод анализа программного кода, который позволяет изучать все возможные пути выполнения программы, используя символьные значения вместо конкретных входных данных. Этот подход позволяет проверять программы на наличие ошибок, находить уязвимости безопасности, а также автоматически генерировать тестовые случаи для обеспечения покрытия кода.

Символьное исполнение становится все более важным в современном мире, так как с ростом сложности программного обеспечения и повышением требований к безопасности становится критически важным обнаружение и устранение ошибок до их попадания в продуктивную среду. Благодаря символьному исполнению, возможно обнаружение скрытых ошибок, которые могут привести к нежелательным последствиям.

Применение решателей проблем SAT (Satisfiability) и SMT (Satisfiability Modulo Theories) в символьном исполнении программ позволяет автоматически исследовать множество возможных путей выполнения программы, анализировать условия ветвления, находить пути выполнения, которые могут привести к ошибкам или уязвимостям. Таким образом, эти решатели значительно упрощают процесс анализа программного кода и играют ключевую роль в задачах символьного исполнения.

Задача поиска ошибки в программе обычно разбивается на задачи проверки истинности специальных предикатов. Для каждого такого предиката работает решатель специальной теории. Задача согласования ответов от решателей специальных теорий записывается в форме КНФ, к которой применяется SAT-решатель. SMT решатель, по сути, реализует взаимодействие между SAT решателем и решателями специальных теорий.

Таким образом, решаемые в процессе сказанного задачи являются комбинаторными и в итоге сводятся к NP-трудной задаче SAT, что, как правило, я вляется узким местом в контексте скорости работы решателей.

Одним из подходов к повышению скорости работы некоторого алгоритма, решающего NP-трудную задачу, является параметризация этого алгоритма: часть настроек сценария его работы задается (как правило, числовыми) параметрами. Для произвольного частного случая исходной задачи можно попытаться подобрать набор значений параметров, дающий лучшую скорость

работы решателя и далее использовать этот набор в аналогичных задачах. Однако этот процесс может быть весьма трудоемким, если число параметров значительно.

Современные SAT и SMT решатели содержат большое количество параметров. Соответственно, полный перебор всевозможных значений параметров представляет из себя вычислительно нецелесообразную задачу. Таким образом, актуальной является задача разработки специальной процедуры автоматического подбора параметров, ускоряющих работу решателей, которые были бы существенно эффективнее полного перебора.

Приведем краткое описание структуры работы. В первой главе происходит введение в предметную область, определяется постановка задач булевой выполнимости и выполнимости формул в теориях, производится обзор техники символьного исполнения программ, формализуется задача подбора параметров алгоритма, а также аргументируется новизна и актуальность работы.

Во второй главе описываются реализованные алгоритмы подбора параметров, описывается метод построения минимизируемой алгоритмами функции, обосновывается выбор алгоритмов.

В третьей главе описана реализация алгоритмов на практике, проведены вычислительные эксперименты, описан результат исследования.

ГЛАВА 1. СИМВОЛЬНОЕ ИСПОЛНЕНИЕ ПРОГРАММ И ЗАДАЧА ПОДБОРА ПАРАМЕТРОВ

В данной главе даётся обзор основных понятий, связанных с SAT, SMT и их применением в задачах символьного исполнения. Также, в этой главе приводится постановка задачи параметризации SAT и SMT решателей.

1.1. Задача булевой выполнимости

Везде далее через множество $\{0, 1\}^n$ обозначен булев гиперкуб размерности n, то есть множеств всех различных битовых векторов длины n.

Булева формула F – это выражение, состоящее из переменных $x_i \in X$, где любая переменная x_i принимает значения из множества $\{0, 1\}$, логических связок $\{\wedge, \vee, \neg\}$ и скобок. Пример:

$$x_1 \wedge (x_2 \vee \neg x_3)$$

Для произвольной булевой формулы F от n переменных и произвольной переменной x_i, входящей в данную формулу, можно естественным образом определить подстановку произвольного значения переменной x_i в F[17]. Результатом подстановки в формулу конкретных значений всех переменных является некоторая константа из $\{0, 1\}$. Таким образом, булева формула от n переменных задает некоторую булеву функцию $f(x_1, ..., x_n) \ : \ \{0, 1\}^n \to \{0, 1\}$.

Литерал – булева переменная или её отрицание. Клоз – дизъюнкция литералов. Пример клоза:

$$x_i \vee \neg x_j \vee x_k$$

В рамках данной работы, рассматривается задача булевой выполнимости для формул, которые находятся в конъюнктивной нормальной форме (КНФ). Конъюнктивная нормальная форма представляется собой конъюнкцию клозов:

$$(x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_4 \vee x_5) \wedge x_6$$

Пусть есть булева формула F и функция $f : \{0, 1\}^n \to \{0, 1\}$, заданная этой формулой. Произвольный набор значений $\alpha = \{\alpha_1, ..., \alpha_n\}$ переменных $x_1, ..., x_n$, для которых $f(\alpha_1, ..., \alpha_n) \ = \ 1$ называется выполняющим набором. Если для формулы F существует выполняющий набор, то F называется выполнимой, в противном случае говорят, что F невыполнима.

Задача булевой выполнимости (SAT) заключается в том, чтобы для булевой формулы найти выполняющий набор или предоставить доказательство, что формула невыполнима. Предполагается, что формула находится в КНФ. В такой постановке SAT – это известная NP-полная задача[3].

Доказательство - это некий сертификат, который можно проверить и убедиться в правильности результата решения SAT задачи. Таким образом, в случае выполнимости формулы сертификатом является выполняющий набор. Если же формула невыполнима, то сертификатом невыполнимости является описание работы некоторого полного алгоритма решения SAT на рассматриваемой формуле.

1.2. SAT решатели

SAT решатель – это программа, которая решает задачу булевой выполнимости. Как уже было сказано ранее, эта задача NP-полна. Таким образом, на данный момент известны лишь алгоритмы, которые в худшем случае решают задачу за экспоненциальное время. Однако, для решения задачи SAT было разработано множество эффективных и масштабируемых эвристик. На практике, это позволяет решателям справляться с формулами, состоящими из десятков тысяч переменных и миллионов клозов, за разумное время.

Для символьного исполнения программ основной интерес представляют полные SAT решатели: такие решатели по произвольной формуле в КНФ должны либо предоставить выполняющий набор, либо доказать невыполнимость данной формулы.

В основе работы SAT решателей зачастую лежат одни и те же алгоритмы, такие как DPLL (Devis, Putnam, Logemann, Loveland) и CDCL (англ. Conflict-driven clause learning). Однако, в практическом смысле, решатели отличаются наличием определенных расширений и эвристик. Таким образом, многие современные подходы к практическому решению задачи булевой выполнимости происходят от алгоритмов DPLL и CDCL и имеют ту же структуру.

Базово, эти алгоритмы выбирают какой-либо литерал, присваивают ему значение 1 и анализируют формулу с данной подстановкой. Если в подстановке литералам значений было выявлено противоречие (чаще это называется конфликтом), то происходит откат (от англ. backtracking) на прошлое состояние, значение конфликтного литерала заменяется на 0. Этот процесс происхо-

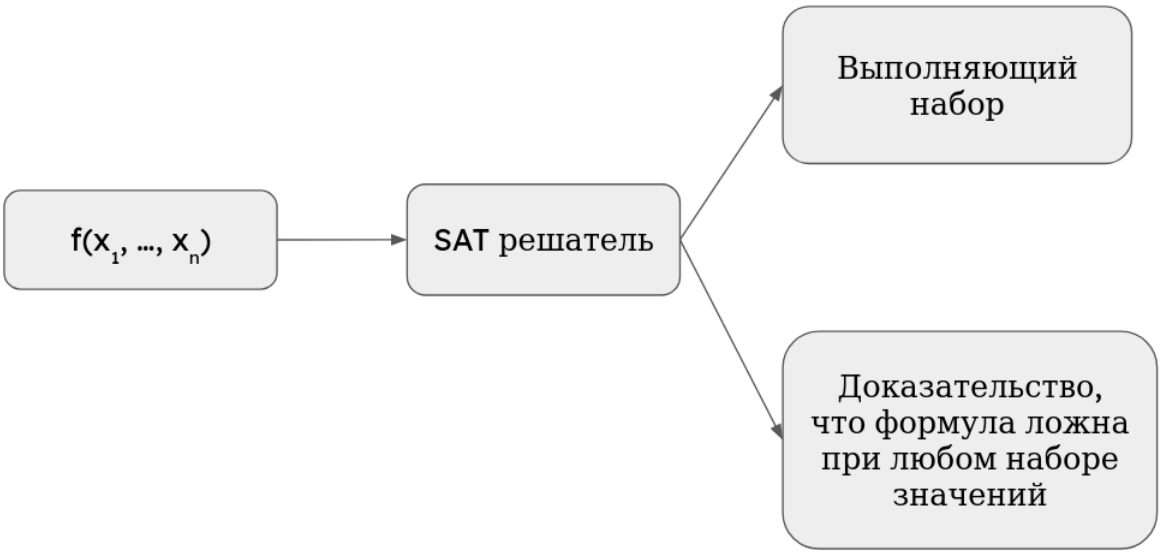

Рисунок 1 – Схема работы полного SAT решателя

дит рекурсивно пока не будет получен выполняющий набор или информация о том, что любая подстановка значений литералов приводит к конфликтам.

Помимо базовой идеи используется множество ее расширений. Это могут рестарты (от англ. restart policies), которые начинают процесс с самого начала в зависимости от того, как ведет себя решатель. Или, например, принцип приоритезации переменных (англ. variable decaying), который заключается в том, что литералам выдаются приоритеты, согласно которым происходит подстановка и перебор значений. Также существуют различные схемы анализа конфликтов и огромное множество различных вспомогательных техник, увеличивающих эффективность базового CDCL и DPLL[2, 9].

Чаще всего, различные расширения и эвристики применяются с целью улучшения производительности решения задач, которые возникают на практике. Это могут быть конкретные классы практических SAT задач, для которых разрабатывается специальный подход, или случайные SAT задачи. Теоретически, алгоритмы имеют экспоненциальную нижнюю оценку, но различные расширения позволяют им быть производительными на практике.

Список некоторых современных SAT решателей:

- CaDiCaL
- Clasp
- CryptoMiniSat
- Gini
- Glucose

- Kissat
- IntelSat

Многие SAT решатели имеют открытый исходный код. Также решатели дают возможность настраивать внутреннее поведение при помощи конфигурации параметров. Можно менять поведение, например, откатов или рестартов изменяя параметры конфигурации решателей, и, таким образом, настраивать решатель под конкретный класс задач с целью увеличения производительности.

Резюмируя сказанное. Задача булевой выполнимости – это NP-полная задача, которая на данный момент эффективно не решается в общем случае. SAT решатели используют сложные алгоритмы с большим количеством эвристик, которые хорошо себя показывают на практике. Существует возможность настраивать параметры этих алгоритмов, тем самым улучшая производительность решателя для конкретных подклассов задач булевой выполнимости.

1.3. Задача выполнимости в теориях

Задача выполнимости формул в теориях (SMT) – расширение задачи булевой выполнимости. Формулы, которые проверяются на выполнимость могут содержать переменные и предикаты из различных теорий. При этом булева логика и задача SAT в определенной выше постановке, также, может быть решена SMT решателем[4].

Примеры теорий:

- Теория битовых векторов, которая содержит в себе битовые операции и предикаты. Например побитовое исключающее или ($b_i \oplus b_j$)
- Теория массивов, которая позволяет моделировать операции над массивами. Например, взятие по индексу ($array[i]$)
- Теория линейной арифметики, в которой есть двухместные предикаты сравнения чисел, умножение, сложение и т.д. В такой теории можно выражать, например, линейные неравенства ($3x \leqslant 10$)
- Теория неинтерпретируемых функций. В этой теории функции и предикаты интерпретируются как абстрактные символы. Так, например, в теории может пользоваться утверждением $x = y \to f(x) = f(y)$. Также, в этой теории есть равенство. Пример формулы: ($f(f(x, y), y) = f(x, y)$)

SMT-формула — это формула в логике первого порядка, в которой функции, предикаты и переменные могут быть интерпретированы особым образом

в зависимости от теории. Переменные в такой формуле произвольные. Они могут быть, например, целыми числами. Тем не менее, задача всё еще состоит в том, чтобы определить выполнима ли данная формула. Более того, в случае выполнимости SMT решатели способны выдать выполняющий набор.

Пример: $3x + 5 \leqslant y$, выполняющим набором может быть $x = 1$ и $y = 10$

Формулу из примера можно интерпретировать следующим образом: Константы – это натуральные числа, переменные имеют значения только из множества натуральных чисел, знак $\leqslant$ – это ”меньше или равно”для натуральных чисел. Таким образом, можно сказать, что в этой формуле используется теория линейной арифметики.

Данная интерпретация не является единственной, поэтому используемые теории, обозначения и способ интерпретации формул устанавливаются заранее и регулируется в SMT-решателе.

Так как формула использует произвольную логику, а не только булеву, выходит, что у SMT-формул выразительная сила больше чем у булевых. Действительно, многое может быть выражено при помощи булевых переменных и логических операций. Однако использование функций и предикатов из теорий значительно упрощает моделирование и значительно сокращает размер формулы, так как аналогичная SMT формуле булева формула может иметь экспоненциально большой размер. Кроме того, может существовать логика, формулы в которой не выразимы в булевой логике.

1.4. SMT решатели

Один из подходов к решению задач SMT – это перевод SMT-формулы в эквивалентную булеву формулу. Так, например, теорию битовых векторов фиксированного размера можно представить в виде булевых формул, представляя 32-битный вектор в виде 32-х булевых переменных, и выражая операции (например, сложение векторов) в виде низкоуровневневых операций с битами. Данный подход известен под названием bitblasting и до сих пор используется некоторыми решателями. Приемущество такой идеи в том, что можно использовать SAT решатель в том виде, в котором он есть изначально. Однако в этом случае SAT решатель может потенциально делать много лишней работы, каждый раз заново доказывая известные факты о теории. Т.е. SAT ре-

шатель не может, например, применить знания о том, что сложение битовых векторов коммутативно ($b_1 + b_2 = b_2 + b_1$).

Для того чтобы иметь возможность делать выводы о формулах в теории на более высоком уровне, был придуман подход похожий на DPLL.

DPLL(T) (Davis-Putnam-Logemann-Loveland with Theory) - это расширение алгоритма DPLL для решения задач выполнимости в теориях. В отличие от классического алгоритма DPLL, который работает только с булевыми формулами, DPLL(T) позволяет работать с SMT-формулами. DPLL(T) представляет собой интеграцию SAT решателя с решателями специфичных теорий.

Решателий теорий очень часто решают NP-трудные задачи. Кроме того, не все теории разрешимы, т.е. не существует алгоритма, который за конечное время корректно определяет верна ли формула. Поэтому SMT решатели работают с ограниченным набором теорий, однако эти теории широко используются в различных задачах и покрывают большое количество практических случаев.

Сначала DPLL(T) абстрагируется от предикатов теории представляя SMT-формулу как булеву, а предикаты как булевы переменные. Алгоритм пытается подобрать значения для этих предикатов, используя SAT решатель. SAT решатель находит некоторый набор подстановок значений для булевых переменных. Далее эти переменные, которые являются предикатами из теорий, проверяются решателями теорий на выполнимость. Решатель теории может получить новую информацию и вывести новые ограничения и добавить их в формулу. После чего SAT решатель может проверить новую формулу на наличие конфликтов. В случае успешной проверки формула упрощается и алгоритм продолжает работать рекурсивно. В случае неудачи в решении теории или проверки SAT решателем происходит откат.

Более детально принцип работы отражен в примере:

$$x + 1 > 0 \wedge (x < 0 \vee x + y > 4) \wedge \neg(x + y > 0)$$

В данной формуле можно провести замену предикатов из теории на булевы переменные и получить следующее:

$$P_1 \wedge (P_2 \vee P_3) \wedge \neg P_4$$

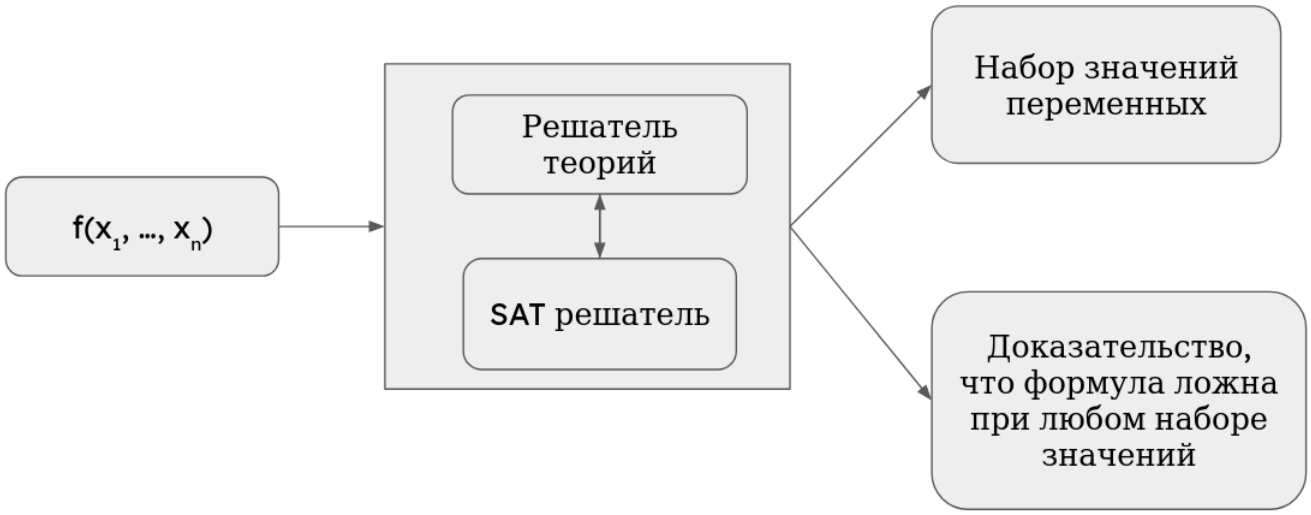

Рисунок 2 – Схема работы SMT решателя

Отсюда можно заключить, что $P_1 \to 1$ и $P_4 \to 0$. Пусть SAT решатель посчитал, что $P_2 \to 1$, тогда необходимо проверить набор $\{P_1, P_2, \neg P_4\}$, который соответствует набору $\{x + 1 > 0, x < 0, \neg(x + y > 0)\}$, на непротиворечивость решателем теории. Решатель теории (в данном примере теории линейной арифметики) получает, что либо P_1 либо P_2 не может быть истинно. Это утверждение добавляется в формулу:

$$P_1 \wedge (P_2 \vee P_3) \wedge \neg P_4 \wedge (\neg P_1 \vee \neg P_2)$$

Добавленный клоз приводит к конфликту, так как значения P_1 и P_2 истинны. В таком случае происходит откат, и алгоритм продолжает свою работу. При этом, измененная формула остаётся в таком же виде. Таким образом, решатель теорий вывел дополнительные ограничения для исходной формулы, что помогает в дальнейшей работе алгоритма. Можно заметить, что новая формула выполнима тогда и только тогда, когда выполнима исходная формула.

Вышесказанное обосновывает значимость роли решателей теорий в процессе работы SMT-решателей. Помимо параметров SAT решателя у SMT решателей существуют параметры специфичные для конкретных теорий, которые также влияют на производительность.

На данный момент есть множество SMT-решателей. Несколько известных Z3, CVC4, Boolector и Bitwuzla широко применяются в автоматическом доказательстве теорем, верификации программного обеспечения, анализа кода и других областях. У некоторых из решателей открыт исходный код. SMT

решатели, как и SAT решатели, дают возможность настраивать параметры внутренних алгоритмов, в том числе и решателей теорий. Также в решателях теорий есть большое количество различных эвристик и схем предобработки формулы, которые можно параметризовывать.

SMT решатели каждый год соревнуются в SMT-COMP на различных наборах SMT-задач. Зачастую решатели там используют набор параметров, который хорошо себя показывает в общем случае, так как принимают участие в соревновании на разных классах задач.

Существует универсальный формат SMT-LIB, в котором записываются SMT-формулы. Также SMT-LIB – это название большой библиотеки содержащей множество SMT-задач разбитых на классы. В данной работе были использованы некоторые материалы из этой библиотеки[13].

Исходя из вышеизложенного, можно заключить, что задачу выполнимости в теориях на данный момент решить эффективно нельзя. Возможность настроики параметров решателей позволяет улучшить производительность решателей на практике в определенных случаях, так как параметры решателей по умолчанию ориентированы на частный случай.

1.5. Символьное исполнение

Символьное исполнение программ – способ анализа кода, который учитывает все возможные пути исполнения программы. Значения входных данных представляются в виде абстрактных ”символьных”значений. Такой анализ программы позволяет для каждой части программы определить при каких входных данных эта часть будет выполняться[1, 6].

Анализаторы, основанные на символьном исполнении, находят применение во многих областях:

- Тестирование программного обеспечения: Анализатор используется для автоматической генерации тестов. Некоторые участки кода могут редко исполнятся за счёт того, что для этого необходимо соблюсти ряд нетривиальных условий. Символьное исполнение поможет сгенерировать тесты с хорошим покрытием автоматически, так как анализатор для каждого участка кода выдаёт набор ограничений, при соблюдении которых код будет выполнен.
- Поиск недостижимого кода: Участки кода, для которых набор ограничений противоречив являются недостижимыми для любого исполнения.

Так как не существует входных данных, которые бы удовлетворяли этому набору ограничений.

- Анализ безопасности и поиск ошибок: Анализатор способен выявлять различного рода уязвимости. Уязвимости можно выразить в виде ограничений, тогда анализатор сможет проверить, нарушаются ли они. Можно находить в программах такие ошибки как ”stack overflow ”undefined behavior”и другие.
- Верификация и доказательство корректности: При помощи анализатора можно проверять свойства программ. Свойства программ – это утверждения вида ”если входные данные удовлетворяют условию P, то выходные данные удовлетворяют условию Q”. Таким образом, можно верифицировать программы.
- Генерация документации и спецификации: Автоматический анализ структуры программы и путей её исполнения позволяет генерировать документацию и спецификацию. Анализатор может сформулировать то, для каких данных функции какой результат ожидать в виде логических формул.

Интерпретатор символьного исполнения обрабатывает код программы, предполагая, что значения входных данных символьные, а не считывает их из потока ввода, как это было бы во время обычного исполнения программы. Далее символьные значения как-то участвуют в дальнейшем исполнении программы, из них строятся новые выражения, они могут встречаться в условии оператора ветвления. Для каждого места в программе анализатор запоминает, что происходило с символьными значениями и выводит набор ограничений для этих значений. Проверка этих ограничений позволяет понять может ли программа исполниться тем или иным образом.

Пусть x – переменная, которой присвоено символьное значение. В программе встречается строка

$$if\ (x == 5)\ \{...$$

В случае выполнения условия будет исполняться первая ветка условного оператора. Для этой части программы анализатор выведет, что символьное значение, которое соответствует переменной x должно быть равно 5 и добавит это ограничение к остальным ограничениям. Когда анализатор закончит работать с первой веткой условия, соответсвующее ограничение будет убрано.

Во время исполнения на входные данные программы могут накладываться различные ограничения. Так, если в исполнении появляется код $y := 4;$, то можно утверждать, что на этом моменте исполнения $y = 4$. Эта информация может быть использована в дальнейшем анализе программы. Также, встречая, например, условный оператор $if(p)$ порождаются два пути с ограничениями p и $\neg p$, соответственно.

Таким образом, произвольный путь представляет собой некоторый набор ограничений. Как показано на примере ниже, не каждый путь может существовать.

```
1   f() {
2       x = read()
3       y = read()
4
5       if (x == 1) {
6           y = 0
7       }
8
9       assert(x + y == 2)
10  }
```

Рисунок 3 – Пример анализа путей выполнения

Рассмотрим пути, в которых проверка на строке 9 проходит успешно. Условие в строке 5 порождает два пути:

1) Так как тело условного оператора выполнено, можем сопоставить пути набор ограничений $x = 1 \wedge y = 0 \wedge x + y = 2$
2) Условие в строке 5 не выполнено, поэтому $x \neq 1 \wedge x + y = 2$

Логическое выражение для пути 1, которое является набором ограничений для этого пути, всегда ложно. Следовательно, путь нереализуем. Проверкой набора ограничений на непротиворечивость занимается SMT решатель, так как набор ограничений представляется в виде SMT-формулы.

Символьное исполнение имеет широкое применение в области анализа программного обеспечения. Популярность этого подхода растёт. На данный

момент имеется большое количество инструментов, реализующих это идею. Вот некоторые из них:

- BE-PUM
- KLEE
- USVM
- Kudzu
- Rubyx
- Verifast

Существуют инструменты для разных языков программирования (Java, C, C++, JavaScript, LLVM). У многих инструментов открыт исходный код. Также, инструменты имеют поддержку различных SMT решателей, которые можно использовать при анализе. Помимо выбора SMT решателя существует возможность вывести все SMT-формулы, на которых приходилось запускать решатель в ходе анализа.

Задача выполнимости в теориях не решается эффективно в общем случае. Поэтому существенной проблемой в скорости работы анализаторов является проверка набора ограничений. Таким образом, ускорение SMT решателя влечет ускорение работы анализатора.

SAT запросы, которые возникают в задачах символьного исполнения имеют определенную специфику. Как правило, они не слишком трудозатраные, но их много. Кроме того, в SMT запросах в основном используются теория линейной целочисленной арифметики, массивов, битовых векторов и строк. Параметры по умолчанию в решателях ориентированы на общий случай, поэтому подбор оптимальных параметров, учитывающих специфику подкласса задач, может ускорить работу решателя на этих подклассах.

Из всего вышеизложенного следует, что подбор параметров SMT решателя может ускорить его работу в задачах символьного исполнения программ и тем самым повысить эффективность анализа современного программного обеспечения.

1.6. Задача подбора параметров

В общем смысле задача подбора параметров алгоритма – это поиск значений параметров, которые оптимизируют работу алгоритма в конкретном случае.

Оптимизировать можно различные характеристики алгоритма, такие как время работы, точность, память и многие другие. В рамках данной работы производится оптимизация времени работы SMT-решателя.

Задача подбора параметров может быть формализована следующим образом: Дано множество параметров $P = \{p_1, ..., p_r\}$, каждый p_i принимает значение из домена D_i, $|D_i| < \infty$. Обозначим пространство поиска за $D = D_1 \times ... \times D_r$(декартово произведение). $F_{A,Q} : D \to \mathbb{R}^+$ - это фитнесс функция, равная времени работы алгоритма A на наборе SMT запросов Q с заданными параметрами. Эту функцию необходимо минимизировать.

SMT решатель – сложная программа с множеством параметров. Вручную подбирать эти параметры очень сложно, поэтому Целью работы ставится разработка алгоритма, который подбирал бы параметры автоматически.

Существуют готовые инструменты для автоматического подбора параметров. Например, SMAC и Optuna, использующие локальный поиск, случайный поиск, Байесовскую оптимизацию и т.д.. Далее в работе будет рассмотрена байесовская оптимизация, которая реализована в интсрументе SMAC[12]. Помимо этого будут разработаны оригинальные алгоритмы подбора параметров и продемонстрировано их преимущество перед SMAC на задачах символьного исполнения.

Следует отметить, что в общем случае какие-либо аналитические свойства функции $F_{A,Q}$ неизвестны. По этой причине в работе используются методы метаэвристической оптимизации.

Метаэвристическая оптимизация – это поиск приближенно оптимальных решений для сложных задач оптимизации. В отличие от детерменированных подходов, которые проделывают строго определенные шаги, метаэвристические методы используют вероятностные элементы и приёмы имитации природных явлений для поиска оптимальных решений. К метаэвристическим оптимизациям относятся, например, эволюционные алгоритмы, генетические алгоритмы, методы симуляции отжига, методы случайного поиска.

Плюсы использования метаэвристических подходов:

- Способность решать задачи, для которых точное вычисление оптимального решения является вычислительно нецелесообразным
- Эффективность в исследовании больших пространств поиска с множеством локальных оптимумов

- Возможность работы с нелинейными, негладкими, многомодальными целевыми функциями

В задачах параметризации ключевую роль играет правильным образом заданная целевая функция $F_{A,Q}$. Пространство, на котором задана функция, для некоторых SMT-решателей может быть большим, поэтому необходимо уменьшить это пространство, оставив параметры, влияющие на результат работы алгоритма для конкретного класса задач. Это поможет сократить вычислительные затраты в работе алгоритма параметризации. Подобное было проделано в рамках данной работы.

Резюмируя сказанное, отметим обоснованность использования алгоритмов метаэвристической оптимизации для подбора параметров ввиду неясности аналитической природы функций, используемых для этой цели. В таких методах функция является черным ящиком и используются различные эвристики для перебора пространства поиска. Такие методы хорошо себя показывают на сложных функциях с неизвестными свойствами. Таким образом, данный метод подходит для параметризации SMT решателей.

1.7. Цель и задачи работы

Целью работы является разработка основанного на метаэвристической оптимизации алгоритма подбора параметров для SMT решателя. Ожидается ускорение работы решателя с параметрами, полученными алгоритмом.

Для достижения данной цели предполагается решение следующих задач:

- Изучить принцип работы SAT и SMT решателей, а, также, параметры, которые можно изменять
- Получить наборы тестов в форме SMT запросов, связанных с символьным исполнением, для параметризации решателей
- Построить фитнес-функции, учитывающие особенности класса решаемых задач
- Разработать алгоритмы оптимизации введенных фитнесс-функций с учетом особенностей областей поиска
- Программно реализовать разработанные алгоритмы
- Провести вычислительные эксперименты

Символьное исполнение программ является актуальным подходом к анализу программного обеспечения. В сфере символьного исполнения SMT

решатели играют важную роль, позволяя эффективно проверять наборы ограничений, описывающих поведение программы, на непротиворечивость и выполнимость формул. Однако, задача выполнимости формул в теориях не решается эффективным образом. Скорость работы SMT решателей во многом зависит от подобранных параметров. Сами решатели никак не настраиваются специальным образом под задачи символьного исполнения. Их настройка по умолчанию хороша для общего случая. Таким образом, разработка алгоритма автоматического подбора параметров для SMT решателей поможет ускорить анализаторы, которые используют актуальный на данный момент метод символьного исполнения программ.

Научная новизна предложенного и реализованного в работе подхода обусловлена применением методов метаэвристической оптимизации для параметризации SAT/SMT решателей на задачах символьного исполнения программ.

Выводы по главе 1

В данной главе был произведен обзор предметной области, были рассмотрены основные понятия, необходимые для работы над заданной темой. Были определены задача булевой выполнимости, задача выполнимости формул в теориях, задача подбора параметров алгоритмов. Также, был описан метод символьного исполнения программ. Также, в данной главе рассматривалась идея метаэвристической параметризации, когда параметризатор ничего не знает об алгоритме, к которому он применяется. Исходя из этого были сформулированы цель и задачи работы, была обоснована актуальность и научная новизна.

ГЛАВА 2. РАЗРАБОТКА МЕТОДОВ ПОДБОРА ПАРАМЕТРОВ И ПОСТРОЕНИЕ ЦЕЛЕВЫХ ФУНКЦИЙ

2.1. Метод Монте-Карло для вычисления целевой функции

Функция $F_{A,Q}(d)$ определенная в первой главе – это суммарное время работы решателя A на всех smt запросах Q с набором параметров d. Следует отметить, что вычсление такой функции – вычислительно дорогая процедура. Поэтому можно вместо вычисления значения $F_{A,Q}$ вычислять некоторую её вероятностную оценку. В данной секции описывается теоретическая основа этого подхода.

Для вычислительной оценки необходимо произвести следующие действия:

1 Зафиксируем N – натуральное число.

2 Выбирем N запросов из Q случайным образом равновероятно.

3 Запустим SMT решатель на каждом запросе и посчитаем среднее.

4 Среднее умноженное на размер исходного множества запросов($|Q|$) объявим значением функции $F_{A,Q}$. Это и есть искомая оценка

Подходы такого типа традиционно относятся к методу Монте-Карло[8]. Аргументация использования данной техники описывается далее.

Зафиксируем некоторый (произвольный) набор значений параметров из D и обозначим суммарное время работы алгоритма A с данными фиксированными значениями на Q через $U_A(Q)$.

Зададим на Q произвольное вероятностное распределение — как правило, имеет смысл задавать равномерное распределение. Будем рассматривать Q как пространство элементарных событий, в котором каждому событию приписана вероятность $\frac{1}{|Q|}$. Это можно сделать, поскольку Q конечно. Также, в силу конечности Q алгебра событий в таком пространстве — это 2^Q, то есть множество всех подмножеств Q.

Свяжем с каждым элементарным событием $q \in Q$ величину $\xi : Q \to \mathbb{R}^+$, равную времени решения задачи q SAT решателем A с некоторым (произвольным) фиксированным набором значений параметров из D. Пусть $Spec(\xi) = \{\xi_1, ..., \xi_d\}$ – множество всех возможных значений, которые принимает величина ξ на элементах Q. Тогда каждому ξ_j, $j \in \{1, ..., d\}$ будет соответствовать сумма вероятностей элементарных событий $q \in Q$, на которых ξ принимает значение ξ_j и, таким образом, случайной величине ξ бу-

дет соответствовать некоторое вероятностное распределение $\{Pr_1, ..., Pr_d\}$ $(Pr_1 + \ ... \ + Pr_d) = 1$.

Рассмотрим математическое ожидание ξ:

$$E(\xi) = \sum_{j=1}^{d} Pr_j * \xi_j$$

Поскольку для произвольного $q \in Q$ $Pr\{q\} = \frac{1}{|Q|}$ (вероятность получить исход q), то суммарное время работы A на множестве Q – это следующая величина:

$$U_A(Q) = |Q| * E(\xi)$$

Последнее соотношение важно, поскольку позволяет аргументированно использовать для оценивания $U_A(Q)$ метод Монте-Карло. А именно, рассмотрим следующие шаги. Выберем из Q равновероятно и независимо N элементов: $\tilde{q_1}, ..., \tilde{q_N}$ и подсчитаем величину

$$\tilde{\xi} = \frac{1}{N} \sum_{j=1}^{N} \tilde{q_j}$$

Утверждается, что величиной $\tilde{\xi}$ можно аппроксимировать $E(\xi)$, увеличивая точность за счет увеличения N. Это есть следствие приводимого ниже варианта неравенства Чебышева[16]:

$$Pr\{(1-\epsilon) * E(\xi) \leqslant \tilde{\xi} \leqslant (1+\epsilon) * E(\xi)\} \geqslant 1 - \frac{Var(\xi)}{\epsilon^2 * N * E^2(\xi)}$$

В последнем неравенстве $Var(\xi)$ – это дисперсия случайной величины ξ, которая может быть весьма большой. В таком случае необходимо подбирать большое N. Обычно $Var(\xi)$ аппроксимируется выборочной дисперсией, которая вычисляется по рассматриваемой выборке.

В статье [**evhardsat**] описывается способ (ϵ, δ)-аппроксимации. По сути, там описывается то, как подобрать для заранее заданных $\epsilon > 0$ и $\delta > 0$ такое N, что следующее неравенство выполнено:

$$Pr\{(1-\epsilon) * E(\xi) \leqslant \tilde{\xi} \leqslant (1+\epsilon) * E(\xi)\} \geqslant 1 - \delta$$

δ задает вероятность, с которой выборочное среднее $\tilde{\xi}$ попадает в интервал от $(1-\epsilon)*E(\xi)$ до $(1+\epsilon)*E(\xi)$.

Утверждается, что достаточно такого N, что следующее неравенство выполнено:

$$N > \frac{Var(\xi)}{\epsilon^2 * \delta * \tilde{\xi}^2}$$

В случае аппроксимации выборочной дисперсией, которая вычисляется по формуле:

$$s^2(\xi) = \frac{1}{N} * \sum_{j=1}^{N} (\xi_j - \tilde{\xi})^2$$

Условие приобретает следующий вид:

$$N > \frac{s^2(\xi)}{\epsilon^2 * \delta * \tilde{\xi}^2}$$

Следует отметить, что вычисление $\tilde{\xi}$ на практике представляет собой запуск SMT решателя на поднаборе SMT-запросов $\{\tilde{q_1}, ..., \tilde{q_N}\}$ с вектором параметров d. Не всегда необходимое для соблюдения неравенства Чебышева N будет существенно меньше $|Q|$. Это зависит от набора тестов Q и текущего d. Для разных d может требоваться разное N и оно может оказатья нецелесообразно большим. А если это значение N как-то перебирается, то может оказаться, что и вовсе потребуется проделать больше вычислений чем в случае, когда метод Монте-Карло не применяется.

Если такое происходит для многих d достаточно часто, то можно увеличить ϵ и δ, тем самым ослабив требования точности оценки в целях экономии ресурсов.

Еще можно выбирать одинаковое значение N, зависящее только от $|Q|$, для всех d, тем самым не задавая точность вообще. Такой подход не подкреплен теоретически, но на практике может показывать неплохие результаты.

Резюмируя, метод Монте-Карло позволяет вычислять вероятностную оценку целевой функции $F_{A,Q}$, потенциально потребляя меньше ресурсов. В применении этого метода очень важно правильно подобрать размер случайного подмножества запросов N, балансируя между точностью вычислений и потребляемыми ресурсами.

2.2. Сведение к задаче псевдобулевой оптимизации

Псевдобулева функция – это любая функция следующего вида:

$$f : \{0, 1\}^n \to \mathbb{R}$$

Задача псевдобулевой оптимизация – это задача оптимизации псевдобулевой функции.

Как было сказано ранее, аналитические свойства оптимизируемой функции неизвестны. В таком случае используются метаэвристические оптимизации. Многие метаэвристические подходы сформулированы для задач псевдобулевой оптимизации.

Если бы все домены D_i параметров у SMT решателей были изоморфны $\{0, 1\}$, то пространство поиска $D = D_1 \times ... \times D_n$ было бы изоморфно $\{0, 1\}^n$ и можно было бы интерпретировать исходную задачу как задачу псевдобулевой оптимизации. Но параметры SMT решателей, в общем случае, данным свойством не обладают. Домены значений параметров могут быть целыми k-битными числами, например.

Сведение исходной задачи к задаче псевдобулевой оптимизации поможет использовать некоторые метаэвристические алгоритмы в исходном виде, которые в теории не подходят для применения к пространствам с произвольными доменами значений параметров.

В данной работе сведение осуществляется при помощи бинарного кодирования. Для этого необходимо:

1) Определить битовое представление для доменов параметров. Это можно сделать построив функцию, которая соотносит параметр с длиной битового представления его домена $lb : P \to \mathbb{N}$, и функцию для каждого параметра p_i, которая соотносит значение из домена с битовым значением $g_i : D_i \to \{0, 1\}^{lb(p_i)}$. Пример:
   - Для кодирования множества $\{a, b, c\}$ достаточно двух бит. Последовательности (00), (01), (10) будут соответствовать a, b и c. Последовательность (11) не соответствует ничему.
   - Для k-битных чисел можно взять их битовое представление
2) Для каждого параметра p_i создать $lb(p_i)$ новых параметров с доменами $\{0,1\}$. Обозначим $\sum_{j=1}^{i-1} lb(p_j)$ за s_i, тогда необходимо создать новые параметры $p'_{s_i+1}, ..., p'_{s_i+lb(p_i)}$. В таком случае для всех p'_i домены будут буле-

выми $D'_i = \{0,1\}$, что соответствует определению задачи псевдобулевой оптимизации.

Суть сведения в построении новой функции $F'_{A,Q} : \{0,1\}^t \to \mathbb{R}^+$ на основе изначальной $F_{A,Q}$, где $t = \sum_{i=1}^{n} lb(p_i)$. Это можно сделать путём обращения функций g_i и применения всех g_i^{-1} к подвектору с позиции $s_i + 1$ по $s_i + lb(p_i)$ вектора $d' \in \{0,1\}^t$. Таким способом получится вектор из изначального пространства D.

Как показано в примере с множеством $\{a, b, c\}$, не всегда для каждого битового вектора существует соответствующее ему изначальное значение параметра. Следовательно, некоторые g_i могут быть обратимы лишь частично.

Обозначим $v[i, j]$ как последовательности элементов вектора v с позиции i по позицию j. Рассмотрим вектора $d' \in \{0,1\}^t$, для которых существует такое i, что $\forall d \in D_i, g_i(d) \neq d'[s_i+1, s_i+lb(p_i)]$. Для таких векторов нельзя получить изначальный вектор d из вектора d'. Пусть значением целевой функции для таких векторов будет $+\infty$, т.е. $F'_{A,Q}(d') = +\infty$. Для остальных векторов используем метод описанный выше с применением частично обращенных g_i.

Такая техника позволяет избежать проблемы отсутствия биекции между D и D' на практике, потому что значение $+\infty$ не повлияет на глобальный оптимум, так как функция времени работы решателя минимизируется.

Таким образом, для того, чтобы иметь возможность использовать некоторые метаэвристические алгоритмы, необходимо произвести сведение исходной задачи оптимизации к задаче псевдобулевой оптимизации. Сведения заключается в построении новой целевой функции $F'_{A,Q}$ на основе исходной $F_{A,Q}$. Способ построения был описан выше.

2.3. Описание эволюционных стратегий

Эволюционные алгоритмы, входящие в область эволюционных вычислений, нацелены на решение задач оптимизации, используя модель естественного отбора и концепции природной эволюции. Они эффективны в случаях, когда формула функции или ограничения могут меняться, и аналитический вид функции неизвестен, что делает применение классических численных методов оптимизации невозможным. Однако эволюционные алгоритмы обычно предоставляют лишь приблизительные решения.

В данной работе используются (1+1)-эволюционный алгоритм в классическом виде, и некоторая его вариация, совмещенная со стратегией локального поиска.

Терминология эволюционных алгоритмов часто заимствована из биологии. Особь представляет собой вариант решения задачи с генотипом, состоящим из генов, хранящихся в хромосомах. Оценка особи производится через функцию приспособленности, оценивающую ее пригодность для решения задачи.

Рисунок 4 – Схема эволюционного алгоритма

Принцип работы эволюционного алгоритма (1+1) состоит в генерации случайной особи, для которой создается ребенок. Обычно, гены ребёнка представленны в виде битового вектора, каждый ген ребенка инвертируется с опре-

деленной вероятностью. Так выглядит мутация. Этот процесс повторяется для каждого ребенка до выполнения условия остановки алгоритма.

Однако в теории особь не обязательно должна быть представлена как битовый вектор, и мутация может быть произвольной. На рисунке изображен общий принцип работы эволюционных алгоритмов. Применительно к поставленной задаче, функция преспособленности – это $F_{A,Q}$, а особь – это вектор из пространства поиска D. Конкретные мутации описаны ниже.

2.4. (1 + 1)-ЭА

В данной работе одним из предполагаемых решений поставленной задачи оптимизации является (1 + 1)-ЭА в классическом виде[5].

Пусть пространство поиска D изоморфно $\{0,1\}^k$ для некоторого k, а $F_{A,Q}$ - функция, которую необходимо минимизировать. Таким образом, необходимо решить задачу псевдобулевой оптимизации. Сведение исходной задачи к задаче булевой оптимизации было описано выше. Можем применить этот подход и минимизировать $F'_{A,B}$, которая выражается через $F_{A,B}$ и битовое кодирование.

Алгоритм:

1) Выбрать начальный вектор $d_0 \in D'$ и вычислить $F'_{A,Q}(d_0)$ и начать с итерации $i := 0$.
2) Получить вектор d'_i применив случайную мутацию к d_i. i – это номер итерации алгоритма. Случайная мутация – изменение каждого значения d_i с вероятностью $\frac{1}{k}$ (k – размерность пространства поиска).
3) Вычислить $F'_{A,B}(d'_i)$.
4) Если $F'_{A,B}(d'_i) \geqslant F'_{A,B}(d_i)$, то $d_{i+1} := d_i$, иначе $d_{i+1} := d'_i$. Выбирается лучший вариант из родителя и ребенка.
5) Увеличить номер итерации $i := i + 1$.
6) Проверить условие остановки. Если останавливаться не надо, то перейти к пункту 2.

```
function OnePlusOneEA(f, d_0)
    d ← d_0                                  ▷ Выбор начального вектора
    f_d ← F_{A,Q}(d)                         ▷ Оценка начального решения
    i ← 1
    while ¬stopCondition do
        d' ← Mutate(d, 1/|d|)                ▷ Мутация
        f_{d'} ← F_{A,Q}(d')                 ▷ Вычисление целевой функции
        if f_{d'} ⩽ f_d then
            d ← d'                           ▷ Принимаем ребенка если он лучше
            f_d ← f_{d'}
        end if
        i ← i + 1
    end while
    return x
end function
```

Критерием остановки алгоритма может служить достижение им определенного числа итераций C или отсутствие улучшения результата на протяжении большого количества итераций.

Важно, что при такой схеме мутации можно использовать разную вероятность изменения компоненты вектора. Данная версия в среднем меняет ровно один элемент вектора, то есть за один шаг не отступает больше чем на 1 по расстоянию Хэмминга от предыдущего вектора. Данная особенность может быть причиной застревания алгоритма в локальных оптимумах и плато. В случае долгого отсутствия улучшения можно делать перезапуск алгоритма.

Начальной точкой может быть случайный битовый вектор. Важно не брать в качестве начального вектора такой d_0, что $F'_{A,B}(d_0) = +\infty$. В таком случае есть риск, что алгоритм застрянет в плато.

Достоинством алгоритма является легкость реализации и малое потребление вычислительных ресурсов на одном шаге. Такой метод может хорошо себя показывать на простых задачах оптимизации, где размерность пространства поиска не очень велика.

2.5. Гибридный алгоритм

Пусть множество значений каждого параметра i конечно ($|D_i| < \infty$). $d \in D$ - вектор пространства поиска, $D = D_1 \times ... \times D_r$. $d\{i \leftarrow x\}$ – обновленный вектор d со значением x в позиции i.

Данная стратегия наследует идею (1 + 1)-ЭА, но использует другой принцип мутации. Также этот способ годится для произвольных доменов пространства поиска D. Сведение к псевдобулевой оптимизации не нужно. Стратегия была разработана автором работы.

Рассмотрим процесс мутации:

1) Дано i – номер какого-то параметра, и оптимизируемая функция $F_{A,Q}$
2) Переберем все значения параметра $x \in D_i$
3) Если $F_{A,Q}(d\{i \leftarrow x\}) < F_{A,Q}(d)$, то $d := d\{i \leftarrow x\}$

Описанный процесс – это локальный поиск по заданному параметру i.

Гибридный алгоритм:

1) Выбрать начальный вектор $d \in D$ и вычислить $f(d)$.
2) Для каждого параметра i с вероятностью $\frac{1}{|D|}$ запустить локальный поиск. Затем, $d := d'$, где d' – вектор полученный локальным поиском по параметру i.
3) Проверить условие остановки. Если останавливаться не надо, то перейти к пункту 2.

```
function HybridAlgorithm(d_0)
    d ← d_0                                   ▷ Выбрать начальный вектор d
    f_d ← F_{A,Q}(d)                          ▷ Вычислить F_{A,Q}(d)
    i ← 0
    while ¬stopCondition do
        for j ← 1 to |d| do
            if RandomProbability() ⩽ 1/|d| then
                f_d, d ← LocalSearch(F_{A,Q}, d, j)   ▷ Локальный поиск для j
            end if
        end for
        i ← i + 1
    end while
    return d                                  ▷ Возвращаем лучшее найденное решение
end function
```

Локальный поиск для целочисленных параметров – это очень трудоёмкий процесс. Поэтому можно применить категоризацию целочисленных значений. Например, взять только значения 10^k, где $0 \leqslant k \leqslant 10$.

Условие остановки совпадает с условием остановки в (1 + 1)-ЭА.

2.6. Байесовская оптимизация

В качестве еще одного подхода была выбрана Байесовская Оптимизация. Байесовская оптимизация - это метод оптимизации, который использует вероятностные модели для эффективного поиска оптимальных параметров функции, которая является дорогостоящей или сложной для вычисления. Модель сама выбирает следующую точку, в которой нужно вычислить функцию f, основываясь на данных о прошлых вычислениях f[15].

P_t – множество состоящее из пар точек и значений целевой функции в них $(d_i, f(d_i))$ после того как алгоритм прошёл t итераций. P_0 - состоит из одной пары начальной точки и значения функции в ней. S - статистическая модель, которая по множеству наблюдений P_t предсказывает следующую точку d, в которой необходимо вычислить значение функции f. Идея заключается в том, чтобы на каждом шаге выбирать новую точку, и с её помощью обновлять множество наблюдений.

1) В начале $t := 0$, $d_{min} = d$, $P_0 = \{(d,\, f(d))\}$, d – начальная точка, d_{min} – найденная алгоритмом оптимиальная точка, t – номер текущей итерации
2) $d' := S(P_t)$ – получение новой точки d' при помощи модели
3) Вычисление $F_{A,Q}(d')$
4) $P_{t+1} := P_t \cup (d',\, f(d'))$
5) Если $F_{A,Q}(d') < F_{A,Q}(d_{min})$, то $d_{min} := d'$
6) $t := t + 1$
7) Если $t < C$, то вернуться к пункту 2

Таким образом, Байесовская оптимизация вычисляет целевую функцию лишь один раз для каждого шага.

Существуют готовые реализации этого метода. Так, в рамках работы была использована библиотека SMAC.

Такой подход тратит мало ресурсов на вычисление целевой функции $F_{A,Q}$. Однако, данный алгоритм сильно зависит от модели, которую он задействует. Кроме того, если целевая функция не облададает какими-либо закономерностями, то алгоритм может проигрывать в скорости поиска минимума.

2.7. Hill Climbing и MVP

Алгоритм Hill Climbing - это простой метод оптимизации, который итеративно улучшает решение путем поиска в окрестности текущего значения функции $F_{A,Q}$. Для вектора $d \in D$ генерируется набор соседей на расстоянии Хэмминга не более чем 1 – $N_1(d)$. Если существует такой $d' \in N_1(d)$, что $F_{A,Q}(d') < F_{A,Q}(d)$, то алгоритм движется в эту точку. Эти шаги повторяются до тех пор, пока не будет достигнуто условие остановки, например, достаточное количество итераций или до тех пор, пока не будет достигнуто плато в окрестности текущей точки.

1) d – начальная точка
2) Если условие остановки выполнено, то закончить процесс
3) Вычисление $N_1(d)$
4) Для каждого $d' \in N_1(d)$ последовательно вычисляется $F_{A,Q}(d')$
5) Если $F_{A,Q}(d') < F_{A,Q}(d)$, то $d := d'$ и вернуться к пункту 2

```
function HillClimbing(d_0)
    d ← d_0                                          ▷ Начальная точка
    f_d ← F_{A,Q}(d_0)
    while true do
        foundBetter ← false
        for d' ∈ N_1(d) do                           ▷ Перебор соседей
            f_{d'} ← F_{A,Q}(d')                     ▷ Вычислить функцию для соседа
            if f_{d'} < f_d then
                d ← d'                               ▷ Обновление точки
                f_d ← f_{d'}
                foundBetter ← true
                break
            end if
        end for
        if not foundBetter then
            break      ▷ Если не найдено лучшего соседа, завершить процесс
        end if
    end while
    return d                                         ▷ Возвращаем лучшее найденное решение
end function
```

Одна из основных проблем алгоритма Hill Climbing - это то, что он может застрять в локальном оптимуме, не суметь добраться до глобального оптимума, если он находится за пределами текущей окрестности. Также алгоритм может останавливаться на плато - области пространства параметров, где все соседние точки имеют одинаковые значения функции.

Достоинством данного подхода является возможность вычислять $F_{A,Q}$ у соседей параллельно.

Для решения проблемы с локальными минумами и плато можно использовать принцип склеивания переменных (Merging Variables Principle, MVP)[11].

Пусть пространство D является булевым. Способ отображения исходного пространства параметров в булево пространство был описан в секции про (1 + 1)-ЭА.

Можно построить разбиение D на r непересекающихся подмножеств X_1, .., X_r. $\bigcup X_i = D$. С каждым множеством $X_p (p \in \{1..r\})$ связывается переменная y_p, домен которой – это декартово произведение доменов всех x из X_p.

Разбиение может быть проделано случайным образом. Причём, это разбиение будет равномерным, т.е. $\forall i\ j, ||X_i| - |X_j|| < 1$.

Существует биекция γ между Y и D. Таким образом, можно построить новую целевую функцию $F'_{A,Q}(x) = F_{A,Q}(\gamma(x)) : Y \to \mathbb{R}^+$.

Каждая переменная y_p является словом над алфавитом $\{0, 1\}$. Естественным образом можно задать метрику Хэмминга.

Глобальный минимум функции $F_{A,Q}$ соответствует глобальному минимуму $F'_{A,Q}$. При этом, нельзя сказать то же самое про локальные минимумы.

Таким образом, имея метрику Хэмминга и γ, можно воспользоваться алгоритмом Hill Climbing на новом пространстве Y. Более того, в случае попадания в локальным минимум можно заново случайным образом перестроить пространство и продолжить выполнять алгоритм.

Hill Climbing использующий MVP был реализован в рамках работы. Идея алгоритма была взята из статьи [11].

Достоинством алгоритма является простота и надежность. Алгоритм за конечное время найдёт какой-то локальный минимум(если он существует). Однако, такой алгоритм может быть медленным в задачах с большой размерностью пространства поиска.

Выводы по главе 2

В данной главе описаны используемые в работе решения задачи подбора параметров, приведен метод оптимизации вычисления целевой функции. Также, описаны достоинства и недостатки используемых алгоритмов.

ГЛАВА 3. ВЫЧИСЛИТЕЛЬНЫЕ ЭКСПЕРИМЕНТЫ

В этой главе содержится описание экспериментов, выбранные для экспериментов SMT запросы, описание выбранного SMT решателя, описание реализации целевой функции с использоваными методами, описанными ранее, результаты экспериментов.

3.1. SMT решатель

Для проведения экспериментов был взят решатель Z3[14] по ряду причин:

- Поддерживает большое количество разных теорий
- Обладает простым интерфейсом
- Много параметров, гибкая конфигурация
- Поддерживается многими анализаторами, которые основаны на символьном исполнении программ

3.2. Тестовые классы

Для проведения экспериментов было взято несколько классов SMT запросов. Все используемые запросы представлены в виде файлов в формате smt2. smt2 – это специальный универсальный формат SMT запросов, который поддерживается большинством решателей. Было взято два класса из SMT-LIB:

- QF_ABV – это SMT запросы без кванторов(QF – от английского quntifier free), которые содержат в себе теорию массивов(A – от анлийского array) и теорию битовых векторов(BV – от английского bit vectors)
- QF_ALIA – запросы без кванторов, содержащие элементы теории массивов и теории линейной арифметики(расшифровывается аналогично, LIA – от английского linear arithmetic)

Особенностью этих классов является комплексность используемых в запросах теорий. Для решателей теорий линейной арифметики и битовых векторо есть много возможностей для настроек. В частности, для решателя линейной арифметики в Z3 имеется 45 параметров. Также, представленные теории достаточно популярны и встречаются часто в задачах символьного исполнения, так как массивы, битовые векторы и арифметика довольно популярны в контексте программирования.

Помимо запросов из SMT-LIB был сформирован класс запросов, которые генерирует интсрумент KLEE в ходе своей работы. KLEE - это анализатор кода программ, разработанный в LLVM. Он использует символьное исполнение[7]. В KLEE есть поддержка Z3, также, в данном инструменте предусмотрена возможность получить все SMT запросы в формате smt2.

В работе KLEE был применен для анализа библиотеки cppcodec, из получившихся в ходе работы инструмента SMT запросов был сформирован третий класс тестов.

Запросы из SMT-LIB могут быть сложнее реальных запросов из задач символьного исполнения. Поэтому для проведения экспериментов были взяты тестовые запросы из всех классов, которые решаются не менее чем за 1 секунду и не более чем за 80 секунд.

3.3. Определение целевой функции

Для определения целевой функции необходимо задать пространство поиска, которое определяется множеством параметров. Так как уменьшение пространства поиска увеличивает точность алгоритмов, для каждого класса были взяты параметры, соответствующие используемым в этих классах теориям.

Для всех классов в пространство поиска были включены общие параметры, отвечающие за работу SAT и SMT решателей. Это параметры, специфичные для алгоритмов CDCL, DPLL и DPLL(T), соответственно.

Из параметров SMT решателя были исключены параметры, которые отвечают за работу с формулами, в которых содержаться кванторы, так как во всех рассмотренных классах формулы безкванторные.

Помимо параметров, связанных с DPLL(T), использовались некоторые параметры, которые отвечают за упрощение формулы. Перед DPLL(T) решатель способен видоизменять формулу, с целью её упрощения. Такие параметры тоже были включены в пространство поиска. Например, булевый параметр $smt.propagate - values$, с активацией которого, решатель пытается заменить идентификатор в формуле константным значением, если ему удается вывести, что значение, соответствующее идентификатору, не меняется.

Для решателей теорий брались только те параметры, которые отвечают за теории, находящиеся в конкретном классе запросов. Так, например, в QF_ALIA не переберались параметры с префиксом $smt.bv$, которые соответствуют решателю теории битовых векторов.

Для метода Монте-Карло необходимо подбирать такое N, которое удовлетворяет условию, описанному в главе 2. Были заданы параметры $\epsilon = 0.2$ и $\delta = 0.1$.

Для подбора N в экспериментах использовалась следующая схема:

а) Начальное значение $N = 2$

б) Выбрать N случайных SMT запросов и посчитать выборочное среднее и выборочную дисперсию

в) Если условие из главы 2 выполнено или текущее значение N больше чем $\frac{|Q|}{3}$, то вернуть в качестве оценки выборочное среднее. В противном случае $N := N * 2$ и перейти к пункту 2

Значение $\frac{|Q|}{3}$ было выбрано в экспериментах, потому что если взять $|Q|$ или $\frac{|Q|}{2}$, например, то в худшем случае решатель будет запускаться на примерно $2|Q|$ или $|Q|$ запросах, соответственно. На практике в проделанных экспериментах слишком часто возникали случаи, когда оптимальное N было не меньше $\frac{|Q|}{2}$.

Из-за небольшого количества вычислительных ресурсов было решено жестко ограничить размер случайного подмножества SMT запросов до $\frac{|Q|}{3}$. Иначе, эксперименты занимали бы слишком много времени. Таким образом, для более быстрого вычисления целевой функции точность пришлось снизить.

3.4. Конфигурация вычислительной платформы

Предложенные алгоритмы были реализованы на языке Python[10]. Для проведения экспериментов использовалась серверная машина с процессором Intel Xeon Gold 6338 2.00 Ггц.

3.5. Постановка экспериментов

Для каждого тестового класса проводился эксперимент. Каждый класс состоял из примерно 120 запросов. В эксперименте происходила параметризация SMT решателя, в качестве запросов, для которых вычислялась целевая функция, использовалась выборка из тестового класса.

- QF_ABV – 20 запросов примерно по 25 секунд каждый
- QF_ALIA – 15 запросов примерно по 35 секунд каждый
- KLEE – 15 запросов примерно по 40 секунд каждый

После нахождения оптимальных параметров для выборки происходила валидация на оставшейся части тестового класса. В результирующей таблице

отражено время работы решателя с подобранными параметрами на валидационном множестве запросов.

Hill Climbing + MVP был запущен с ограничением в 12000 вычислений целевой функции. Перебор соседей осуществлялся в 4 потока. Также, было выставлено ограничение на не более 5 перестроений пространства поиска. Если это ограничение нарушалось, то происходил перезапуск алгоритма из новой начальной точки. Выбиралась лучшая точка с учетом всех перезапусков.

Остальные алгоритмы запускались 4 раза параллельно с ограничением на 3000 вычислений на каждый с различным значением seed для генератора псевдослучайных чисел. Если в течении 1000 шагов алгоритм не улучшал текущий результат, то производился перезапуск из новой начальной точки. Лучшая точка выбиралась с учетом нескольких запусков и перезапусков алгоритма.

Начальная точка выбиралась случайном образом во всех случаях. В качестве значения $+\infty$ для целевой функции использоволось значение 10000. Функция возвращала время в секундах. Лимит времени для решения одного SMT запроса составлял 200 секунд. Если существовал хотя бы один запрос, который превышал этот лимит, то значением целевой функции назначалось 10000. Соответственно, начальная точка выбиралась случайным образом, но так, что значение функции в ней было меньше 10000 секунд.

Для проверки статистической значимости результатов использовался тест Вилкоксона для двух выборок. Тест Вилкоксона является непараметрическим статистическим методом, который используется для проверки различий между двумя независимыми выборками X и Y. Он основан на ранжировании значений из обеих выборок и вычислении статистики, которая затем сравнивается с критическими значениями для определения статистической значимости различий. В данной работе X_i из первой выборки – это время работы SMT запроса с параметрами по умолчанию, Y_i – это, соответственно, время работы запроса с параметрами, которые были найдены алгоритмом.

Далее можно видеть таблицы для каждого эксперимента.

Алгоритм	Ускорение, %	Время, сек (с параметрами по умолчанию 1701)	P-value (Тест Вилкоксона, α = 0.05)
Гибридный	9	1535	0.008
(1 + 1)-EA	7	1578	4e-5
HC + MVP	12	1496	0.002
SMAC	3	1653	0.002

Рисунок 5 – Эксперимент QF_ABV

В запросах, содержащих теории битовых векторов и массивов, лучшее ускорение получилось у алгоритма Hill Climbing (12%). Статистический тест показал значимость ускорения во всех алгоритмах.

Алгоритм	Ускорение, %	Время, сек (с параметрами по умолчанию 2481)	P-value (Тест Вилкоксона, α = 0.05)
Гибридный	11	2201	0.0001
(1 + 1)-EA	7	2300	0.0019
HC + MVP	14	2137	4e-7
SMAC	3	2405	0.04

Рисунок 6 – Эксперимент QF_ALIA

Для линейной арифметики получилось добиться значимого ускорения в 14% (Hill Climbing + MVP). Для эволюционного алгоритма (1 + 1) и байесовской оптимизации не было получено статистически значимого результата.

Алгоритм	Ускорение, %	Время, сек (с параметрами по умолчанию 1690)	P-value (Тест Вилкоксона, α = 0.05)
Гибридный	15	1425	0.006
(1 + 1)-EA	7	1559	0.16
HC + MVP	19	1363	0.001
SMAC	4	1636	0.24

Рисунок 7 – Эксперимент KLEE

Hill Climbing + MVP и гибридный алгоритм показали большой процент ускорения для реальных задач символьного исполнения, которые были сгенерированы при помощи анализатора KLEE. Остальные результаты хуже и не являются статистически значимыми.

Выводы по главе 3

В этой главе были описаны использованные в вычислительных экспериментах класс SMT запросов, описан способ задания целевой функции с учетом особенностей использованных классов тестов, предоставлены результаты эксперементов.

Эксперименты показали, что подбор параметров с использованием методов метаэвристической оптимизации, примененный к SMT решателям в задачах символьного исполнения программ, может дать значимый прирост в скорости.

В данный момент ведется работа над актом о внедрении.

ЗАКЛЮЧЕНИЕ

В данной дипломной работе была разработана методика параметризации SMT решателей для задач символьного исполнения программ. Были выбраны и реализованы несколько алгоритмов параметризации, основанные на метаэвристических методах, были проведены вычислительные эксперименты с реализованными алгоритмами. Были описаны способы построения и использования целевых функций. Производилось сравнение с существующим популярным подходом – байесовской оптимизацией.

Проведенные исследования показали, что разработанные алгоритмы позволяют снизить время работы, затрачиваемое SMT решателем во время анализа программ при помощи техники символьного исполнения.

Значимость работы заключается в том, что разработанные алгоритмы параметризации SMT-решателей могут быть использованы для разработки более эффективных инструментов анализа программного обеспечения.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1 *Baldoni R., Coppa E.* A Survey of Symbolic Execution Techniques [Электронный ресурс]. — 2018. — URL: https://arxiv.org/pdf/1610.00502.

2 *Biere A., Fröhlich A.* Evaluating CDCL Restart Schemes [Электронный ресурс]. — 2019. — URL: https://www.semanticscholar.org/paper/Evaluating-CDCL-Restart-Schemes-Biere-Fr%C3%B6hlich/6eb4e435378e11b43ca64ff434131c3c91a2cb41.

3 *Biere A., Heule M.* Handbook of Satisfiability: Volume 185 Frontiers in Artificial Intelligence and Applications. — 2009.

4 *De Moura L. , Bjorner N.* Satisfiability modulo theories: introduction and applications [Электронный ресурс]. — 2011. — URL: https://dl.acm.org/doi/pdf/10.1145/1995376.1995394.

5 *Droste S., Jansen T., Wegener I.* On the Analysis of the (1+1) Evolutionary Algorithm // Theoretical Computer Science. — 2002.

6 *King J.C.* Symbolic execution and program testing. Communications of ACM. Vol. 19. — 1976. — 385–394.

7 KLEE Symbolic Virtual Machine [Электронный ресурс]. — URL: https://github.com/klee/klee.

8 *Metropolis N., Ulam S.* The Monte Carlo method // Journal of the American Statistical Association. — 1949. — 335–341.

9 *Ouyang M., Chvatal V.* Implementations of the DPLL Algorithm [Электронный ресурс]. — 1999. — URL: https://www.researchgate.net/publication/242384772_Implementations_of_the_DPLL_Algorithm.

10 Python 3.12.3 documentation [Электронный ресурс]. — URL: https://docs.python.org/3/.

11 *Semenov A.* Merging variables: one technique of search in pseudo-Boolean optimization [Электронный ресурс]. — 2019. — URL: https://arxiv.org/abs/1908.00751.

12 SMAC3 Documentation [Электронный ресурс]. — URL: https://automl.github.io/SMAC3/main/.

13 SMT-LIB [Электронный ресурс]. — URL: https://smt-lib.org/.

14 Z3 Theorem prover [Электронный ресурс]. — URL: https://github.com/Z3Prover/z3.

15 *Елистратова Е.* Подбор гиперпараметров [Электронный ресурс]. — URL: https://education.yandex.ru/handbook/ml/article/podbor-giperparametrov.

16 *Феллер У.* Введение в теорию вероятностей и ее приложения. Т. 1. — 1970.

17 *Чень Ч., Ли Р.* Математическая логика и автоматическое доказательство теорем. — 1983.